

AI 驱动运维平台 工程开工文档

版本 0.1 | 审核稿 | 本文档通过后作为平台实施、验收与多 Agent 协作的共同基线。

项目状态	当前说明
阶段	架构讨论完成，尚未进入代码实施阶段
首个验证能力	光模块健康分析，先用 Fixture 与模拟场景验证平台链路
开工条件	项目负责人审核并明确批准进入实施阶段
约束	第一阶段严格只读；Demo 与 Standalone 以 Docker Compose 为部署基线

本文件如何使用

它不是一次性需求说明，而是施工基线。任何功能、代码、测试、文档或 Agent 任务均不得与本文件的已确认决策冲突；如需改变，应先通过架构决策记录审批。

0. 文档控制与阅读方式

本文档将内容分为三类：已确认表示可作为实施约束；默认方案表示在无新决定时采用；后续再议表示不阻塞开工，但不得被默认实现为生产承诺。

类别	处理规则	示例
已确认	实施任务必须遵守；修改需记录 ADR 并由项目负责人批准。	严格只读、Zabbix 为首个 Connector、独立 Test Agent 验收
默认方案	按此执行；如实现中证据表明不合适，由 Project Lead 提出变更。	Python + FastAPI、React + TypeScript、Docker Compose
后续再议	可建立接口预留，但不可假设已支持或宣称已完成。	Kubernetes、写操作、真实生产数据源、多模型路由

建议阅读顺序

角色	优先阅读章节	目标
项目负责人	1、2、5、10、12、13、15	确认目标、边界、阶段与验收方式
Project Lead	全部	据此拆分任务并维护实施基线
Delivery Agent	3、4、6、8、9、10	按模块、契约和工程标准交付
Test Agent	4、5、10、11、14	独立建立测试设计与完成判定

目录

- 1. 项目愿景与成功标准
- 2. 范围、非目标与设计原则
- 3. 总体架构与模块边界
- 4. 核心数据模型与平台契约
- 5. 安全、权限与数据治理
- 6. 部署 Profile 与运行环境
- 7. 对话、工作台与能力目录
- 8. 技术规范与工程约束
- 9. Git、代码管理与项目管理
- 10. 多 Agent 协作工作流
- 11. 测试、验收与功能完成包
- 12. 分阶段实施计划
- 13. 项目负责人的职责与沟通方式
- 14. 风险、变更与决策机制
- 15. 开工检查清单与附录

1. 项目愿景与成功标准

AskLily 是面向 GPU 数据中心及相关运维环境的 AI 驱动运维平台。它以受控的数据查询、可追溯的领域结论和可交互的工作台为核心，使运维人员能用对话方式理解资产、事件、指标、链路与健康状态。

最终目标

目标	完成后的可观察结果
自然语言运维入口	用户在授权范围内以中文问题发起查询，获得有数据来源、时间范围和限制说明的结论。
可审计的工作台	复杂问题能从对话切换到工作台，自动定位资源、筛选条件和时间范围；页面数据可刷新、分页、追溯。
可扩展的平台	新增光模块、GPU、RDMA 等能力时，复用统一身份、Scope、Tool、ViewContext、测试和能力目录。
私有化可用	可在本地、内网或私有化环境部署；不以公网或 Kubernetes 作为首期前提。
可信交付	功能是否完成由明确验收标准、独立测试证据和负责人确认决定，而非由 Agent 的主观描述决定。

首期成功定义

- 建立空平台骨架而非堆叠业务页面：身份 Scope、契约、能力注册、Agent Tool、工作台状态、审计与测试基础均可复用。
- 完成“光模块健康”纵向切片：模拟数据 -> 健康规则 -> Tool -> 对话结论 -> ViewContext -> 工作台 -> 独立测试验收。
- 将该能力标记为“Demo 已验证”，不宣称真实 Zabbix 或 Production 已验证。

成功不等于功能很多
首期价值在于证明新能力可以沿同一条治理链路安全、可测、可审计地加入平台。

2. 范围、非目标与设计原则

首期范围

范围	首期要求
用户体验	简洁的 Chat 模式；可切换的双栏 Workspace 模式；工作台状态可由对话继续驱动。
数据	Fixture 与规则驱动模拟数据；为 Zabbix 只读 Connector 预留标准接口。
权限	基础角色、项目与资源 Scope；每次查询在服务端执行范围过滤。
能力	光模块健康为首条纵向能力；能力目录呈现已验证、规划中和限制。
部署	Demo 和 Standalone 的 Docker Compose Profile；离线和内网安装可行。

首期明确非目标

非目标	原因
自动修复、设备配置下发、工单写入	首期严格只读，先消除权限、审批与误操作风险。
Kubernetes 与高可用集群	当前没有开发环境；先以可重复的 Compose 部署证明平台能力。
真实生产环境接入承诺	尚无真实环境；真实 Connector 验收必须单独进行。
一次性实现全部运维领域	以纵向能力切片推进，防止平台壳、接口和数据语义失控。

设计原则

01. 契约先行

任何跨模块行为先定义版本化契约，再实现前端、后端、Tool 与测试。

02. 范围先于智能

先根据身份生成 Scope；模型只能在允许的 Tool 与范围内工作。

03. 事实与结论分离

观测、事件、规则、健康结论分别建模；Agent 不能把推测表达为事实。

04. 按证据扩展

性能、组件拆分、Go 模块或生产基础设施均由容量和压测证据驱动。

05. 可验收优先

功能必须在真实或模拟场景下可重复验证，而不只是页面能打开。

3. 总体架构与模块边界

架构起点采用模块化单体 + 异步 Worker。逻辑模块边界现在固定，部署可先保持简单。未来只有在性能、独立扩容或隔离需求有证据时才拆为独立服务。

逻辑全景

交互层	编排与治理层	领域与数据层
Chat UI Workspace UI Capability Center	Identity + Scope Agent Orchestrator Tool Registry View Registry Audit	Domain Query Services Connectors PostgreSQL VictoriaMetrics Object Storage

请求路径

用户问题 -> Chat UI -> Agent Orchestrator -> 已授权 Tool -> Domain Query Service -> Connector / 数据库 -> 结构化结果 -> 文字结论 + ViewContext -> Workspace
Agent 负责理解与组织，领域服务负责查询与判断，工作台负责交互与展示。任何一层都不能绕开 Scope。

模块职责

模块	必须负责	不得负责
Web	会话展示、工作台状态、可访问性、调用平台 API	直连 Zabbix、绕开权限、自行解释业务健康规则
Agent	问题理解、Tool 选择、结果归纳、生成 ViewContext	直接查询数据库、扩大用户 Scope、制造未验证事实
Tool	输入校验、权限复核、调用领域能力、审计	承载前端显示细节、信任 Agent 传入的权限范围
Domain	标准化查询、规则计算、结构化结果	持有模型 Prompt、耦合外部厂商字段
Connector	连接外部源、映射、数据质量标注	决定最终健康结论、直接服务前端

语言策略

默认采用 React + TypeScript 构建前端，Python + FastAPI 构建平台 API、领域服务、Connector 和 Agent 编排。Go 只用于经容量测量证实存在高吞吐瓶颈的独立模块；Rust 暂不纳入首期规划。语言选择不得成为跨模块重写的理由。

4. 核心数据模型与平台契约

平台只统一跨模块必须共享的最小语义。厂商字段、领域算法和具体 UI 细节保留在各自模块中，避免公共模型无限膨胀。

对象	最小含义	关键字段
Resource	可管理、授权和展示的对象	id、type、project_id、site_id、name、lifecycle_state、source_refs
ResourceRelation	资源之间的可追溯关系	source_id、target_id、relation_type、observed_at、source_ref
Observation	单次状态或指标事实	resource_id、metric_key、value、observed_at、source_ref、quality
Event	已去重的异常和状态演进	id、fingerprint、resource_ids、severity、status、first_seen_at、last_seen_at
HealthAssessment	面向用户的健康判断	resource_id、health、reason_codes、evaluated_at、evidence_refs、rule_version
Scope	由身份生成的允许数据边界	project_id、site_ids、cluster_ids、resource_types、actions

事实与结论的边界
RX 光功率为 -18.7 dBm 是 Observation；连续 37 分钟低于阈值是 Event；当前健康为严重是 HealthAssessment。Agent 只能引用这些有来源的结果，不能把模糊推测包装成事实。

核心契约

契约	作用	版本与拥有者
API Contract	Web、外部客户端与 API 的请求和响应语义	语义版本；API / Domain 共同维护
Tool Contract	Agent 可调用能力、参数、权限要求与返回结构	语义版本；Agent / Domain 共同维护
View Contract	工作台允许的 View、字段、筛选与状态	语义版本；Web / Domain 共同维护
Connector Contract	外部源映射、质量、时间语义与错误类型	语义版本；Connector 维护
Capability Manifest	能力、Tool、View、Profile、验证状态的聚合事实	版本化；Project Lead 审核

5. 安全、权限与数据治理

平台安全的第一原则是：权限范围由服务端根据身份生成，前端和 Agent 不拥有扩大范围的权力。前端隐藏菜单不是安全控制。

基础角色

角色	主要职责	首权限限
Platform Admin	平台、项目、能力与部署策略治理	创建项目、配置能力、查看全局审计
Project Admin	项目内用户、数据源、策略治理	项目范围管理和配置
Operator	日常运维分析	按 Scope 查询、分析、确认告警；首期无写操作
Auditor	审计与复核	只读查看审计和授权范围内结果

多层权限执行

位置	必须执行的控制
会话建立	身份认证、项目上下文、基础角色加载
Agent	仅暴露授权 Tool；不允许模型传入的 Scope 覆盖服务端范围
Tool API	重新鉴权、参数校验、审计请求与拒绝
Domain Query	将 Scope 转换为强制过滤条件；汇总数也在范围内计算
数据层	项目隔离、行级或标签过滤、敏感字段脱敏
结果返回	范围复核、来源与新鲜度提示、敏感数据脱敏

数据治理与模型 Provider

- 项目之间采用独立数据平面；首期可按“一套部署实例对应一个项目”落地，Production 再演进多项目控制面。
- 模型 Provider 以抽象接口接入。真实数据进入模型前需经过 Scope 过滤与脱敏策略；未来可支持本地模型。
- 任何生产密钥、真实客户数据、未脱敏日志、原始大规模指标数据不得进入 Git。
- 第一阶段严格只读。写操作、自动修复、设备配置与工单修改必须在后续独立架构决策后才能加入。

6. 部署 Profile 与数据策略

三种部署模式共享同一份业务契约与能力定义，差异只体现为配置、数据源、容量和基础设施能力。

Profile	用途	首期能力	不承诺的内容
Demo / Developer	本地开发、开源演示、功能验证	Fixture、模拟数据、可选真实只读 Connector、完整功能协议	高可用、生产容量、真实环境已验证
Standalone	普通私有化部署	RBAC、审计、备份、升级、正式只读业务功能	多区域高可用和大规模多项目治理
Production	公有云或大型客户私有云	扩扩容、灾备、多项目治理、生产运行控制	首期不实施，后续独立立项

运行环境

首期以 Docker Compose 作为 Demo 与 Standalone 基线。它必须能在本地、内网和离线环境被可重复安装。Kubernetes 暂不考虑，但配置结构不能将未来迁移封死。

数据真实性分层

级别	数据来源	作用
L0 Fixture	小型固定样本	单元、契约和确定性回归测试
L1 Synthetic Scenario	规则驱动模拟数据	覆盖故障组合、权限与异常路径
L2 Calibrated Synthetic	真实统计特征校准后的合成数据	容量、分布和更真实的场景验证
L3 Sanitized Replay	脱敏事故回放	复现真实事故形态
L4 Live Read-only	真实系统只读连接	Connector 兼容、字段差异、延迟和真实空值验收

首个 Connector 约束
Zabbix 是首个真实数据源方向。当前没有真实环境，因此光模块能力先以 L0 与 L1 验证；后续接入 Zabbix 时，必须单独完成 L4 Connector 验收，不能把 Demo 通过表述为生产可用。

7. 对话、工作台与能力目录

Chat 和 Workspace 不是两个独立产品，而是同一会话的两种布局状态。用户在工作台中仍可继续对话；对话的后续结果可以继续改变右侧视图。

交互规则

场景	平台行为
简单查询	Chat 给出短结论、数据范围、来源和新鲜度；无需打开工作台。
复杂或多对象查询	Chat 给出摘要和“在工作台查看”入口；用户可切换到双栏布局。
工作台内追问	左侧会话继续调用 Agent；新的 ViewContext 可更新右侧视图、筛选、时间范围或资源焦点。
页面交互回流	用户从表格或图表选择资源时，其标识可进入会话上下文，供下一轮对话引用。

ViewContext 协议

ViewContext 是 Agent 和 Web 之间的结构化跳转协议。它表达“打开什么视图、按什么范围和条件展示”，但不包含用于直接渲染的自然语言结论，也不承担授权。

字段	意义	控制规则
view_id	已注册工作台能力，例如 optic_health	只能引用 View Registry 中的条目
scope	项目、站点、集群和资源边界	服务端以身份权限覆盖或收窄
filters	该视图公开的筛选条件	按 View Contract 校验
focus_resource_ids	高亮、定位或展开的资源	不提供额外授权
time_range	统一时间语义	文字、表格和趋势必须一致
query_id	可选的短期查询快照	支持审计、复现和昂贵查询复用

能力目录

面向管理者的权威入口是 Capability Catalog，而不是 Tool 列表或孤立 Wiki。它由 Capability Registry 生成，展示名称、状态、Profile、数据源、权限、View、Tool、最近验证结果、限制和负责人。Wiki 或仓库文档用于记录设计原因、接入手册和排障知识。

8. 技术规范与工程约束

以下规范用于让不直接审查代码的项目负责人仍能获得可验证的工程质量。所有自动化检查应由 CI 执行，不能依赖某位 Agent 的主观承诺。

领域	标准
TypeScript	严格类型检查；前端不直接引用外部厂商数据；使用 API / View Contract 类型。
Python	类型标注、Pydantic Schema、清晰错误模型；Domain 与 Connector 不能以隐式字典约定沟通。
API	OpenAPI 或等价版本化契约；所有响应带可追溯 request / query 标识。
日志与审计	结构化日志；高敏感查询、权限拒绝、Tool 调用和导出操作必须可审计。
错误处理	区分无数据、数据延迟、权限拒绝、数据源失败、规则无法评估；Agent 不得将失败伪装成结论。
配置	配置与代码分离；敏感值仅经环境或秘密管理注入；提供 .env.example 而非实际密钥。
依赖	锁定版本、自动漏洞扫描、升级有测试证据；不允许为了临时任务引入不可维护依赖。
性能	先测量再优化。所有大范围查询需分页、时间范围和上限；高基数指标受策略约束。

数据保留原则

数据类型	策略
资产与配置	保存当前状态、关键变化和源引用；不为每次同步重复写完整快照。
事件	按 fingerprint 去重，记录首次/最近发生、次数、严重度、状态演进和关联资源。
时序指标	采集过滤、标签预算、热数据短保留、降采样趋势、长期业务摘要；严重事件可保存高分辨率窗口。
Agent 与审计	长期保存元数据和摘要；完整 Prompt、Tool Response、会话内容按敏感级别和策略选择性保留。

9. Git、代码管理与项目管理

项目采用一个私有 GitHub 单仓库。单仓库便于共同版本化契约、业务实现、测试场景、部署配置和文档；不因逻辑模块划分而过早拆为多个仓库。

推荐目录逻辑

目录	用途
apps/web	Chat、Workspace、Capability Center 前端
services/api, services/agent, services/worker	平台 API、Agent 编排和异步任务
packages/contracts, packages/domain	公共契约和领域模型
connectors	Zabbix、Prometheus、CMDB 等适配器
tests	单元、契约、集成、E2E、Fixture、Scenario、测试报告
infra	Compose Profile、CI、部署和运行配置
docs	架构、能力说明、ADR、运行手册

分支与合并规则

- main 必须代表当前可运行、可测试的基线，禁止直接修改。
- 一个能力或明确任务使用一个短生命周期分支，例如 feature/CAP-OPTIC-HEALTH。
- 多 Agent 并行必须使用独立 Git Worktree 或不重叠的文件范围；禁止在共享工作目录中并发修改同一文件。
- 提交应是可解释的原子改动。每次完整任务在测试后推送 GitHub，不采用“周期性随意上传”。
- 契约、权限、数据库迁移、Connector、部署和写操作属于高风险改动，必须额外审查。

GitHub 项目管理

使用 Issue 记录可验收任务，使用 Project 看板记录状态，使用里程碑管理阶段。代码、测试报告和文档均通过任务编号关联。推荐状态：Idea -> Architecture agreed -> Ready -> In development -> Test verification -> Owner acceptance -> Released。

10. 多 Agent 协作 workflow

首期采用最少必要角色，避免形成管理成本高于产出的“虚拟组织”。每个角色按权限和责任被约束，不以 Agent 自述作为完成依据。

角色	职责	默认不可做
项目负责人	产品目标、架构决定、范围确认、最终验收和发布授权	无需逐行审代码，不直接协调每个开发 Agent
Project Lead	拆任务、冻结范围、协调契约、安排集成、生成完成包	不得以口头结论替代测试证据；不得自行扩大范围
Delivery Agent	在允许目录内实现前端、后端、Agent 或 Connector，并完成自测	不得随意修改公共契约、RBAC、其他能力或生产配置
Test Agent	测试设计、独立验收、回归、缺陷证据和报告	默认不得修改业务代码、产品契约和权限策略
按需 Reviewer	安全、文档或性能专项审查	不应在所有小任务中自动启动

任务协议

字段	必须内容
任务标识	任务或能力编号、标题、责任角色、目标分支和基线提交
范围	允许修改目录、依赖契约版本、明确非目标
验收	业务验收项、权限场景、异常场景、回归范围、性能或真实数据要求
约束	不可修改项、不可执行操作、数据和密钥限制、是否允许网络或外部系统访问
完成条件	测试证据、文档更新、缺陷状态、报告位置、交接信息

并行与串行

可并行	应串行或受 Lead 协调
互不重叠的 Web、Domain、Fixture 任务；独立测试设计；文档草稿	公共契约、RBAC、数据库迁移、部署配置、同一工作台协议、同一文件的变更

不越权规则
未获得“进入实施阶段”授权前，任何 Agent 均不得创建、修改或运行项目代码。实施中，任何超出任务协议的需求、权限或数据访问都必须返回 Project Lead 和项目负责人确认。

11. 测试、验收与功能完成包

“开发 Agent 表示完成”只表示自测完成。完整功能只有在独立 Test Agent 通过、文档齐备且项目负责人接受后才算业务完成。

测试分层

层级	主要验证	触发
L0 静态	格式、lint、类型、Schema、依赖与配置	每次提交 / CI
L1 单元	纯函数、规则、数据转换、组件、权限判断	每个开发任务
L2 契约	Web / API、Tool / Registry、Agent / Response、Connector / Domain	模块与集成任务
L3 集成	API + 数据库、Worker、Agent + Tool、Connector Emulator	每个纵向切片
L4 E2E	用户问题到 Agent、数据、结论和工作台的完整路径	每个纵向切片
L5 真实只读	真实 Connector 的版本、字段、延迟与空值	Connector 验收和里程碑
L6 容量	资产量、活跃序列、并发、权限开销和查询延迟	里程碑和 Production 前

功能完成包

项目负责人默认只阅读一页功能完成包。它必须链接到详细测试报告和功能说明，但不能只提供链接而不展示结论。

必填项	管理者应看到的内容
状态	可验收、条件验收或不可验收
本次实现	能力新增内容和不包含内容
测试摘要	验收、权限、E2E、回归、真实数据测试的通过、失败和未测状态
风险与限制	哪些限制可接受，哪些将进入后续任务
建议	Project Lead 对接受、退回或延后的明确建议
证据	基线提交、功能说明、完整测试报告、演示或截图位置

验收颜色

状态	最低条件
绿色 - 可验收	验收项通过，无 Critical / High 缺陷，相关测试与文档齐备。
黄色 - 条件验收	能力可用但真实数据、性能或明确范围仍有限制；限制已登记并由负责人接受。
红色 - 不可验收	核心项失败、权限风险、关键场景未测、证据不可复现或范围未说明。

12. 分阶段实施计划

阶段不以时间承诺衡量，而以可验证交付物和验收门槛衡量。每个阶段结束后由 Project Lead 形成阶段报告并向项目负责人请求下一阶段授权。

阶段	目标	关键交付物	退出条件
P0 开工基线	建立可控施工环境	Git / GitHub 治理、目录、任务模板、ADR、CI 骨架	负责人批准工程基线
P1 平台骨架	验证共性链路	身份 Scope、契约、Tool / View / Capability Registry、基础审计、Chat / Workspace 壳	基础契约与静态检查可运行
P2 光模块纵向切片	验证端到端平台效果	Fixture、健康规则、查询、Tool、对话、ViewContext、工作台、E2E	Demo 验收为绿色或明确黄色
P3 Zabbix 只读接入	验证真实系统兼容性	Connector、字段映射、质量报告、L4 验收	真实只读 Connector 验收通过
P4 Standalone 硬化	使私有化部署可交付	Compose Profile、备份、审计、升级、运维 文档、容量基线	Standalone 验收通过
P5 能力扩展	复用平台新增 GPU / RDMA 等能力	新能力纵向切片及回归库	每项均按完成包关闭
P6 Production 规划	在证据基础上规划生产治理	HA、K8s、多项目、灾备、性能方案	单独架构审批，不由首期默认 触发

首期光模块能力验收重点

- 用户提问能调用受授权的光模块 Tool，并给出数据范围、时间和来源说明。
- 复杂问题能生成合法 ViewContext，工作台带入站点、健康筛选、时间范围和焦点资源。
- 健康规则对正常、RX 低、TX 低、温度高、恢复、数据缺失、重复告警均有可测行为。
- 用户只能看见授权 Scope；汇总数、搜索建议和工作台结果均不得泄露其他范围。
- Demo 数据验证通过后明确标记限制；真实 Zabbix 验收作为后续独立里程碑。

13. 项目负责人的职责与沟通方式

项目负责人不需要审核每一行代码。应以业务目标、架构决策、风险接受和验收结果管理项目。Project Lead 是唯一日常汇报入口，但其结论必须有测试与文档证据。

你需要做的事

时点	你的决定
开工前	审核本文档，确认是否进入实施阶段；确认主技术方向、只读边界和首个能力。
能力启动	确认功能目标、用户价值、非目标和验收标准；不需要拆代码任务。
架构变更	仅对影响范围、成本、安全、数据或部署的变更作决定。
能力关闭	阅读一页功能完成包，选择接受、条件接受、要求演示或退回。
发布前	确认目标 Profile、已知限制和风险接受范围；决定是否发布。

你如何向 Project Lead 提需求

推荐表达方式
“我希望用户 X 在场景 Y 中获得结果 Z。必须满足 A、B；暂时不做 C。请先给我能力任务说明、验收条件、风险和计划，不要直接写代码。”

Project Lead 必须向你汇报

汇报类型	内容	频率
开工请求	目标、边界、契约影响、任务拆分、验收和风险	每个完整能力启动前
风险升级	阻塞原因、可选方案、推荐方案、所需决定	发现高风险时立即
阶段进展	已完成、进行中、下一步、风险、无需你处理的细节	按里程碑或约定节奏
功能完成包	状态、测试摘要、限制、建议、证据链接	每个完整能力结束时

你不需要做的事

- 不需要选择每个库、目录或具体类名。
- 不需要阅读完整测试日志，除非完成包提示风险、失败或你想深入。
- 不需要同时管理多个 Agent；遇到冲突由 Project Lead 协调并升级决策。

14. 风险、变更与决策机制

大型项目不可避免会遇到新信息。正确做法不是固守初始方案，也不是任意改动，而是将影响架构的变化记录为可审计决定。

需要 ADR 的变更

类别	示例
安全与权限	改变项目隔离方式、引入写操作、允许模型访问新敏感数据
数据与合规	改变保留周期、接入真实数据、允许数据出网、引入新数据类别
核心契约	Resource / Event / ViewContext / Tool 的破坏性变更
部署与容量	引入 Kubernetes、拆独立服务、改变灾备或运行 Profile
技术栈	引入 Go 服务、替换数据库、引入关键外部依赖

ADR 最小模板

字段	要求
问题	为什么当前方案不能满足需求
候选方案	至少列出可行方案及其代价
决定	选择何方案、谁批准、何时生效
影响	代码、数据、部署、测试、迁移与回滚影响
验证	怎样证明决定有效，何时复审

首期风险登记

风险	当前处理
没有真实 Zabbix 环境	Demo 只用 Fixture / Scenario 验证；L4 真实只读验收独立计划。
多 Agent 造成冲突	任务协议、独立 Worktree、契约所有者、Project Lead 集成门禁。
模型结论不可靠	限定 Tool、引用证据、错误类型、Agent Eval 和人工验收。
初期复杂度过高	模块化单体、Compose、最少角色、单一纵向切片，证据驱动扩展。
负责人不审代码	以完成包、测试报告、文档、Git 基线和验收门禁替代逐行审查。

15. 开工检查清单与附录

项目只有在以下开工门禁通过后才进入实施阶段。任一未通过项应明确为阻塞或条件，而不是默认忽略。

开工门禁	状态
项目负责人审核并批准本工程开工文档	待审批
确认 Python + FastAPI、React + TypeScript 为主技术栈	待审批
确认 Demo / Standalone 使用 Docker Compose，首期不做 Kubernetes	已确认
确认第一阶段严格只读	已确认
确认首个能力为光模块健康，Zabbix 为未来首个真实 Connector	已确认
创建私有 GitHub 仓库和保护 main 的规则	实施阶段 P0
建立任务模板、ADR、Capability Manifest 和完成包模板	实施阶段 P0
确定第一个 Capability Brief 的验收标准	实施阶段 P0

附录 A - Capability Brief 模板

CAP-XXX 标题

用户价值：谁在什么场景解决什么问题。

范围：包含内容。

非目标：本次明确不做的内容。

依赖：数据源、资源模型、契约和 Profile。

验收：功能、权限、异常、E2E、回归、真实数据和性能要求。

风险：已知限制和需要项目负责人决定的事项。

附录 B - 功能完成包模板

功能完成包

功能与状态：绿色 / 黄色 / 红色。

本次实现：已完成和未包含。

测试摘要：静态、单元、契约、集成、E2E、真实数据、回归。

风险与限制：可接受与不可接受项。

建议：接受、条件接受、退回或演示。

证据：Git 基线、功能说明、测试报告、演示路径。

下一步

项目负责人审核本文档后，以一句明确指令授权：“工程开工文档已批准，进入实施阶段，先执行 P0 开工基线。”在此之前，项目保持讨论阶段，不创建或修改项目代码。